

Cockpit Control / Vico SW Manual

Version 0.1

Omer.Kannar@elbitsystems-us.com

December 5, 2024

Date	Description of Change	Author
12/5/2024	Initial draft created	Omer Kannar

1 Visual layouts

- a) Main Container: Defines the main screen size, background color and grid density.
- b) Panels Container: A container placed over the main container and contains a list of panels.
- c) Layout: List of panels' containers that predefined.
- d) Panel
- e) Component: Anything that is supposed to interact with the user (Switch, Knob, Display, etc..)

2 Configuration files

- a) settings.json
- b) PanelConfiguration.json
- c) Layouts.json
- d) Panels.json
- e) Component_xxx.json

3 Basic data interface

Each component is defined by its own basic data. The basic data is an interface which allows the user to control the component's behavior, position, size and connectivity to simulation. The interface is defined as:

```
1 interface BasicData {
2   type: Type;
3   backend: {
4     key: string;
5     dbsimProps?: {
6       stationName: string;
7       blockName: string;
8       elementName: string;
9       elementType: ElementType;
10      enumMapping?: any;
11    }
12  };
13  component: {
14    debugMode?: boolean;
15    isClickable: boolean;
16    position: {
17      imgScale: number;
18      imgWidth: number;
19      imgHeight: number;
20      posLeft: number;
21      posTop: number;
22      zIndex: number;
23    };
24    imageProps: {
25      imageDefault: string;
26      additionalImageData?: any;
27    };
28    clickProps?: {
29      clickBoundsHeightFactor: number;
30      clickBoundsWidthFactor: number;
31      mapping: {
32        mapPressPull1?: string;
33        mapPressPull2?: string;
34        mapTop?: string;
35        mapBottom?: string;
36        mapLeft?: string;
37        mapRight?: string;
38      };
39    };
40    knobProps?: {
41      rotation: any;
42    };
43    analogProps?: {
44      conversion: any;
45    };
46    stringProps?: {
47      maxStringLength: number;
48    };
49    blinking?: {
50      color: string;
51    };
52    logger?: {
53      display: boolean;
54    };
55  }
56 }
```

3.1 type

Can be one of the following:

- "static" – The component does not change its image. Typically, this type is used to allow the user to click on it without triggering any other specific behavior.
- "stateN" – This is the most common type. The designer is responsible for updating the image whenever new data is received from the simulation. The designer should have all the images related to this specific component. For example, a 3-position switch (up/center/down) is a stateN type, and the designer should have images for each position: up, center, and down.
- "knobInteger" – This type is used for knobs with N (2 or more) rotation positions. The designer should have an image of the knob, and based on data from the simulation, the knob will rotate by X degrees from its default position (negative X degrees indicates counterclockwise rotation, while positive X degrees indicate clockwise rotation).
- "analogRotation" – This type is very similar to "knobInteger" except that this type receives analog input and using linear interpolation, convert the data to rotation. The knob is then rotated to the desired position.
- "analogVerticalTranslation" – Received analog values from the simulation and responsible to move the image up and down according to linear conversion.
- "analogHorizontalTranslation" – Similar to "analogVerticalTranslation" while in this case – the image moves from left to right according to linear conversion.
- "number" – Display a number received from the simulation. The designer is responsible for providing a list of images corresponding to each number and the maximum length of the number.

3.2 backend

This object defines all the properties of the component related to the simulation.

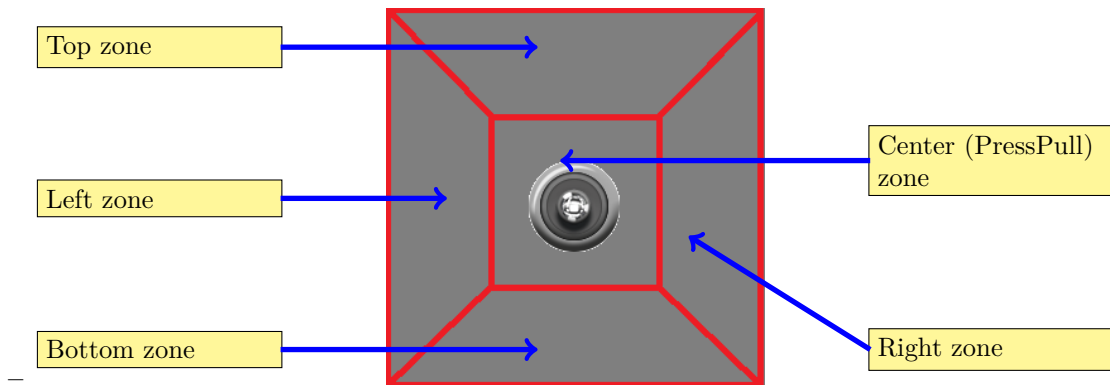
- "key" - Logical name to describe the component. Will be used for logging purposes.
- "dbsimProps" - Defines all dbsim properties
 - "stationName" - Host name
 - "blockName" - LRUs block name connected to component
 - "elementName" - LRUs element name connected to component
 - "elementType" - Type of the element. Can be one of the following: "Double", "Float", "Integer", "Boolean" or "String"
 - "enumMapping" - With assumption that most of the components are 4 bytes and will be enumerations, this object defines the enum mapping that was configured with DBSim. The structure of this object should be as follows:

```
1 {  
2   "Enum 1" : value1,  
3   "Enum 2" : value2,  
4   "Enum 3" : value3,  
5   ...  
6 }
```

3.3 component

This object defines all the functional and visual properties of a component.

- “debugMode” - [boolean] Defines whether a component is in debug mode or not. Debug mode gives the user additional data about the component behavior.
- “isClickable” - [Boolean] Defines whether a component is clickable or not.
- “position” - Defines all properties about component’s position and size
 - “imgScale” - [number] [% / 100] Defines the scale of an image (1 = 100%, 2 = 200%, 0.5 = 50%...)
 - “imgWidth” - [number] [px] Defines image width
 - “imgHeight” - [number] [px] Defines image height
 - “posLeft” - [number] [px] Defines horizontally image position from the left- top corner
 - “posTop” - [number] [px] Defines vertically image position from the left- top corner
 - “zIndex” - [number] Defines the “layer” of an image (If there several images in the same position, this property defines which image is going to be stack. The higher zIndex - The higher the image is)
- “imageProp” - Defines all properties about component’s image\s
 - “imgDefault” - Defines the path of the default image
 - “additionalImageData” - Defines all other images associated to the component. (In case there are several images, this object will defines them all)
- “clickProps” - Defines all properties about component’s click behavior
 - “clickBoundsHeightFactor” - [number] [% / 100] Defines the click zone height of a component (from original component height) (1 = 100%, 2 = 200%, 0.5 = 50%...)
 - “clickBoundsWidthFactor” - [number] [% / 100] Defines the click zone width of a component (from original component width) (1 = 100%, 2 = 200%, 0.5 = 50%...)
 - “mapping” - Defines all zones where click is available and the behavior of each zone (See Appendix 1 - Examples)



According the requested behavior, the values should be one of the following: “TOGGLE_INCREASE”, “TOGGLE_DECREASE”, “EXPLICIT VALUE”, INCREASE \ CW or DECREASE \ CCW

- * “mapPressPull1” - Center square, Defines as 1/3 area from the outer square
- * “mapPressPull2” - Reserved - not used
- * “mapTop” - Top trapezoid
- * “mapBottom” - Bottom trapezoid
- * “mapLeft” - Left trapezoid

* “mapRight” - Right trapezoid

- “knobProps” - Defines all properties of knob component when type is “knobInteger”
 - “rotation” - The structure of this object should be as follows (See Appendix 1 - Examples):

```
1 {  
2   "Key 1" : Rotation value1,  
3   "Key 2" : Rotation value2,  
4   "Key 3" : Rotation value3,  
5   ...  
6 }
```

- “analogProps” - Defines all properties of the analog component when type is “analogRotation”, “analogVerticalTranslation” or “analogHorizontalTranslation”
 - “conversion” - Assuming linear conversion, the structure of this object should be as follows:

```
1 {  
2   "Key 1" : Conversion value1,  
3   "Key 2" : Conversion value2,  
4   "Key 3" : Conversion value3,  
5   ...  
6 }
```

- “stringProps” - Defines all properties of the string component when type is “string”
 - “maxLength” - Defines the maximum length of a string
- “blinking” - Defines all blinking properties
 - “color” - [Explicit value (“black”, “green”, ...), RGB, RGBA, HEX, HSL, HWB, CMYK, NCOL]
- “logger” - Defines whether the component will appear in log or not.
 - “display” - [boolean]

4 Appendix 1 - Examples

4.1 stateN

The following is applied for component with 2 to unlimited states.

Assume 3 states switch with the following images:



And these images should populate the following panel:



In addition, Let's assume that the values from DBSIM are as follows:

Enumerator Values (3)				
Enum Value	Raw Min Value	Raw Max Value	Enum Name	Range
0	0	0	OFF	InLimits ▾
1	1	1	ANTISKID	InLimits ▾
2	2	2	BRAKE	InLimits ▾

In this case the images mapping will be as follows:

```
1 "component": {
2   ...
3   "imageProps": {
4     "imageDefault": "pathDefault",          // Path to default image (Usually one of the
                                              // paths below
5     "additionalImageData": {
6       "OFF": "pathDown",                    // path to switch down
7       "ANTISKID": "pathMiddle",             // path to switch middle
8       "BRAKE": "pathUp"                     // path to switch up
9     }                                       // close "additionalImageData"
10  }                                       // close "imageProps"
11 }                                       // close "component"
```

Now, according to requirements, the click behavior should be defined. If explicit behavior is required, each click zone should contain the explicit value. In this case:

```
1 "component": {
2   ...
3   "clickProps": {
4     "mapping": {
5       "mapTop": "BRAKE",                     // will inject the value "BRAKE
6       "mapPressPull1": "ANTISKID",          // will inject the value "ANTISKID
7       "mapBottom": "OFF"                    // will inject the value "OFF"
8     }                                       // close "mapping"
```

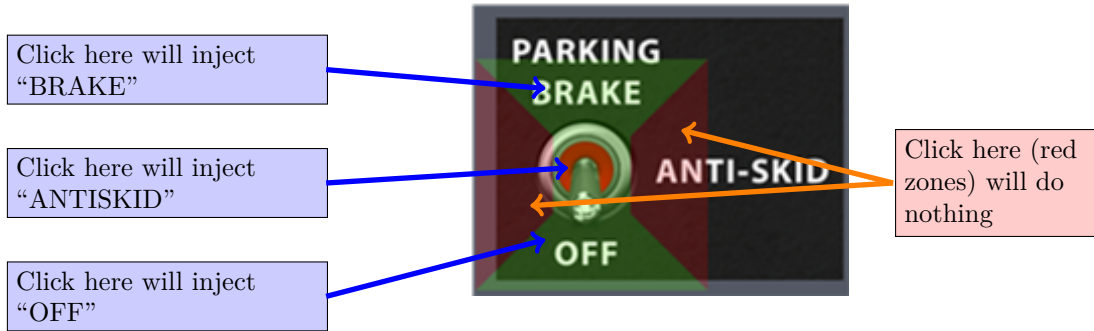


```

9   }                                     // close ""clickProps
10  }                                     // close ""component

```

And the following will result:



Click on the upper trapezoid will inject the value "BRAKE", click on the lower trapezoid will inject the value "OFF" and click on the center square will inject the value "ANTISKID". It was possible to define the left and right trapezoids by mapping "mapLeft" and "mapRight" but in this case it isn't necessary.

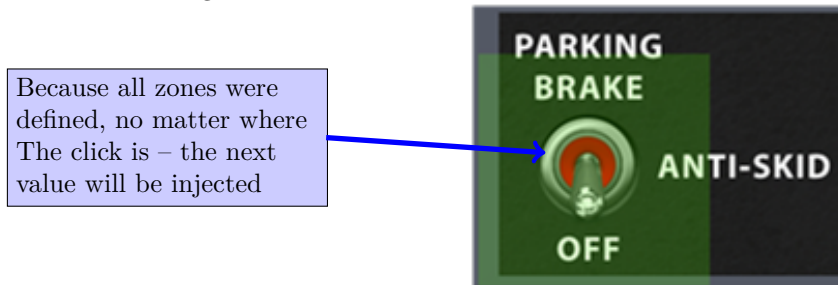
If toggle behavior is required, each click zone should contain the value "TOGGLE_X" (Where X = INCREASE or DECREASE). In case a toggle increase is required, the value should be as follows:

```

1  "component": {
2    ...
3    "clickProps": {
4      "mapping": {
5        "mapTop": "TOGGLE_INCREASE", // will toggle to next value
6        "mapPressPull1": "TOGGLE_INCREASE", // will toggle to next value
7        "mapBottom": "TOGGLE_INCREASE", // will toggle to next value
8        "mapLeft": "TOGGLE_INCREASE", // will toggle to next value
9        "mapRight": "TOGGLE_INCREASE", // will toggle to next value
10     }
11   } // close "mapping"
12 } // close "clickProps"
    // close "component"

```

And the following will result:



The logic will be "OFF" -> "ANTISKID" -> "BRAKE" -> "OFF" -> "ANTISKID" -> ...

In case a toggle decrease is required, the value should be as follows:

```

1  "component": {
2    ...
3    "clickProps": {
4      "mapping": {
5        "mapTop": "TOGGLE_DECREASE", // will toggle to previous value
6        "mapPressPull1": "TOGGLE_DECREASE", // will toggle to previous value
7        "mapBottom": "TOGGLE_DECREASE", // will toggle to previous value
8        "mapLeft": "TOGGLE_DECREASE", // will toggle to previous value
9        "mapRight": "TOGGLE_DECREASE", // will toggle to previous value
10     }
11   } // close "mapping"
12 } // close "clickProps"
    // close "component"

```

And the following will result:

Because all zones were defined, no matter where The click is – the previous value will be injected



The logic will be “OFF” -> “BRAKE” -> “ANTISKID” -> “OFF” -> “BRAKE” -> ...

It is also possible to combine behaviors so one or more zones will implement explicit value, and other zones will implement toggle behavior.

4.2 knobInteger

The following is applied for components with one image.

Assume a knob with the following images:



And these images should populate the following panel:



In addition, Let's assume that the values from DBSIM are as follows:

Enum Value	Raw Min Value	Raw Max Value	Enum Name	Range
0	0	0	BIT	InLimits
1	1	1	1	InLimits
2	2	2	2	InLimits
3	3	3	3	InLimits
4	4	4	4	InLimits

In this case we should include only the knob image and image mapping will be as follows:

```

1 "component": {
2   ...
3   "imageProps": {
4     "imageDefault": "path",           // Path to knob image
5   }                                   // close "imageProps"
6 }                                   // close "component"

```

In addition, “knobPrpos” should be defined as well:

```

1 "component": {
2   ...
3   "knobPrpos": {
4     "rotation": {
5       "BIT": -90,    // "BIT" is -90deg from default knob position
6       "1": -55,     // "1" is -55deg from default knob position
7       "2": -15,     // "2" is -15deg from default knob position
8       "3": 25,      // "3" is 25deg from default knob position
9       "4": 69,      // "4" is 69deg from default knob position
10    }               // close "rotation"
11  }                 // close "knobPrpos"
12 }                 // close "component"

```

Notice that positive rotation (+Xdeg) is CW rotation and negative rotation (-Xdeg) is CCW rotation.

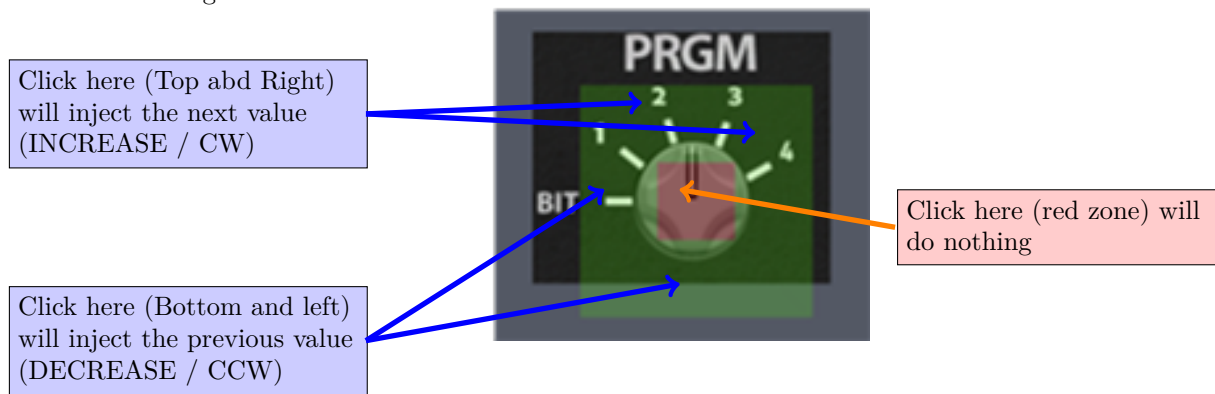
Now, according to requirements, the click behavior should be defined. If explicit behavior is required, each click zone should contain the explicit value. In this case:

```

1 "component": {
2   ...
3   "clickProps": {
4     "mapping": {
5       "mapTop": "INCREASE", // will inject the next value (can be also the value "CW")
6       "mapRight": "INCREASE", // will inject the next value (can be also the value "CW")
7       "mapBottom": "DECREASE", // will inject the previous value (can be also the value "CCW")
8       "mapLeft": "DECREASE" // will inject the previous value (can be also the value "CCW")
9     } // close "mapping"
10  } // close "clickProps"
11 } // close "component"

```

And the following will result:



To summarize, click on the right and top trapezoids will implement the following logic: "BIT" → "1" → "2" → "3" → "4" (more clicks will do nothing – the value will remain "4"). click on the left and bottom trapezoids will implement the following logic: "4" → "3" → "2" → "1" → "BIT" (more clicks will do nothing – the value will remain "1").

If toggle behavior is required, each click zone should contain the value "TOGGLE_X" (Where X = INCREASE or DECREASE). In case a toggle increase is required, the value should be as follows:

```

1 "component": {
2   ...
3   "clickProps": {
4     "mapping": {
5       "mapTop": "TOGGLE_INCREASE", // will toggle the next value
6       "mapRight": "TOGGLE_INCREASE", // will inject the next value
7       "mapBottom": "TOGGLE_INCREASE", // will inject the next value
8       "mapLeft": "TOGGLE_INCREASE", // will inject the next value
9       "mapPressPull1": "TOGGLE_INCREASE" // will inject the next value
10    } // close "mapping"
11  } // close "clickProps"
12 } // close "component"

```

And the following will result:

Because all zones were defined, no matter where the click is – the next value will be injected



In this case, the logic will be “BIT” -> “1” -> “2” -> “3” -> “4” -> “BIT” -> “1” -> ...
In case a toggle decrease is required, the value should be as follows:

```
1 "component": {  
2   ...  
3   "clickProps": {  
4     "mapping": {  
5       "mapTop": "TOGGLE_DECREASE", // will toggle the previous value  
6       "mapRight": "TOGGLE_DECREASE", // will inject the previous value  
7       "mapBottom": "TOGGLE_DECREASE", // will inject the previous value  
8       "mapLeft": "TOGGLE_DECREASE", // will inject the previous value  
9       "mapPressPull1": "TOGGLE_DECREASE" // will inject the previous value  
10    } // close "mapping"  
11  } // close "clickProps"  
12 } // close "component"
```

And the following will result:

Because all zones were defined, no matter where the click is – the previous value will be injected



In this case, the logic will be “BIT” -> “4” -> “3” -> “2” -> “1” -> “BIT” -> “4” -> ...